Guión / acordeón de presentación

Proyecto Apertura — DBSCAN paralelo con OpenMP

José y Natalia

Martes 20 de octubre de 2026

Cómo usar este documento

Es un **guión de apoyo**, no para leer en voz alta frente al grupo: es para tenerlo abierto a un lado (celular, hoja impresa) y ver de un vistazo qué sigue y qué decir en cada parte. Repartan quien dice qué bloque **antes** de entrar.

Duración sugerida total: 8–10 minutos (ajusten según el tiempo que les den).

Orden de la presentación

#	Bloque	Tiempo	Quién
1	Qué problema resolvemos	1 min	
2	El algoritmo (Paso 1 / Paso 2)	1.5 min	
3	Las 3 versiones del código	2 min	
4	Demo en vivo (compilar + correr)	1.5 min	
5	Resultados y gráficas	2 min	
6	Conclusiones	0.5 min	
7	Preguntas	lo que sobre	ambos

---

1) Qué problema resolvemos (1 min)

Decir: > “Implementamos un detector de puntos *core* y ruido/outlier, estilo DBSCAN, pero con el algoritmo ingenuo de dos pasos que pide el proyecto —no el DBSCAN completo con clusters, solo clasificar cada punto—. Lo hicimos en tres versiones: una serial de referencia y dos paralelas con OpenMP, para comparar dos estrategias distintas de paralelizar el mismo algoritmo.”

Mostrar (opcional): la imagen del PDF del proyecto con el círculo rojo/verde/azul/amarillo (core vs. ruido).

---

2) El algoritmo — Paso 1 / Paso 2 (1.5 min)

Decir, con sus propias palabras, apoyándose en esto:

- **Paso 1:** para cada punto, cuento cuántos vecinos tiene a distancia ≤ `epsilon`. Si tiene al menos `min_samples` vecinos, es **punto core** (de primer orden).
- **Paso 2:** los puntos que no fueron core en el paso 1 se revisan otra vez: si están a distancia ≤ `epsilon` de **algún** punto core, se vuelven **core de segundo orden**. Si no, se quedan como **ruido/outlier**.
- Es **O(n²)**: cada punto se compara contra todos los demás. A propósito no usamos árboles espaciales (k-d tree) porque el punto del proyecto es comparar formas de paralelizar ese costo cuadrático, no optimizar el algoritmo en sí.

Parámetros usados: `epsilon = 0.03`, `min_samples = 10` (los mismos que usa la libreta oficial `DBSCAN_noise.ipynb`; fijos en todos los experimentos).

---

3) Las 3 versiones del código (2 min)

Decir:

> "Las tres hacen exactamente el mismo cálculo y dan exactamente el mismo resultado —lo comprobamos—. La diferencia es solo cómo reparten el trabajo entre hilos."

- `dbscan_serial` — un solo hilo, es la referencia (el 1.0x contra el que medimos todo).
- `dbscan_p1` **(matriz indivisible)** — le decimos a OpenMP "reparte estos n puntos entre los hilos" con `#pragma omp parallel for`, y es **OpenMP el que decide**, en caliente, qué punto le toca a cada hilo (`schedule(dynamic)`, porque unos puntos cuestan más que otros).
- `dbscan_p2` **(dividir por k)** — nosotros **dividimos a mano** los n puntos en `k` bloques, cada bloque se procesa aparte y guarda su resultado en un buffer propio, y al final hay un paso explícito de **unir (merge)** esos bloques en el resultado global. Esto importa porque un punto de un bloque puede necesitar enterarse de un punto core que encontró **otro** bloque —por eso el merge va completo antes de empezar el Paso 2.

Si preguntan "¿por qué no hay condiciones de carrera?" → cada hilo/bloque escribe solo en su propio índice o buffer; nunca dos hilos escriben en la misma casilla.

---

4) Demo en vivo (1.5 min)

Comandos exactos a teclear (desde la carpeta `dbscan_proyecto/`):

```
make
./bin/dbscan_serial -i datos/puntos_20000.csv -o /tmp/salida.csv -e 0.03 -m 10 -t 1
./bin/dbscan_p1     -i datos/puntos_20000.csv -o /tmp/salida.csv -e 0.03 -m 10 -t 4
./bin/dbscan_p2     -i datos/puntos_20000.csv -o /tmp/salida.csv -e 0.03 -m 10 -k 8 -t 4
```

Cada comando imprime una línea `TIEMPO ...` con el tiempo de cómputo. **Muéstrenla** —es la prueba de que el programa realmente corre y mide.

Si quieren algo visual: abran uno de los CSV de `datos/` o `graficas/0X_*.png` directo en la compu.

---

5) Resultados y gráficas (2 min)

Números clave a decir de memoria:

- Corrimos **2,320 experimentos** (8 tamaños de n × 10 repeticiones, con todas las combinaciones de hilos y de k).
- *Speedup* promedio: **1.62x con P1**, **1.48x con P2** (máximos de hasta 2.56x) —ambas por arriba del 1.5x que pide el proyecto en la mayoría de configuraciones—.
- Pico máximo: **3.62x** (P2, n=80,000, k=64, 8 hilos).
- El *speedup* deja de crecer después de 4 hilos porque el equipo de pruebas **solo tiene 4 núcleos físicos reales** (es una máquina virtual de 4 vCPU); los hilos 5–8 comparten esos mismos 4 núcleos.

Mostrar: `graficas/01_speedup_vs_hilos.png` (la más clara) y `graficas/05_comparacion_p1_p2.png`.

Si preguntan "¿cuál versión es mejor?" > "Son equivalentes en la mayoría de los casos. P1 es más simple y no depende de ningún parámetro extra. P2 puede empatar o superar a P1 si se elige bien el valor de k —en nuestros datos, cuando k es al menos el doble del número de hilos—, pero si k es muy chico, P2 pierde contra P1."

---

6) Conclusiones (30 seg)

> "Las dos estrategias de paralelización cumplen el objetivo: ambas superan 1.5x de speedup. P1 es la opción más robusta y sencilla; P2 da más control (el parámetro k) y puede rendir mejor si se calibra, a cambio de más complejidad en el código (el paso de merge)."

---

7) Preguntas — top 8 que probablemente les hagan

Respuestas cortas; el detalle completo está en el PDF de "Explicación técnica".

1. **¿Por qué epsilon=0.03 y min_samples=10?** — Son los mismos valores que usa la libreta oficial `DBSCAN_noise.ipynb` del curso, para esa escala de puntos (rango ~[0,1.3]); se dejaron fijos en todos los experimentos para que la única variable fuera el tamaño del problema, k y los hilos.
2. **¿Por qué no hay condiciones de carrera (race conditions)?** — Cada hilo escribe solo en su propia posición del arreglo (P1) o en su propio buffer local (P2); nunca dos hilos tocan la misma celda al mismo tiempo.
3. **¿Qué es la "barrera implícita"?** — Al terminar un `#pragma omp parallel for`, OpenMP espera a que **todos** los hilos acaben antes de seguir. La usamos para asegurar que el Paso 2 no arranque hasta que el Paso 1 (y, en P2, el merge) esté completo en todos los hilos/bloques.
4. **¿Por qué el speedup no sigue subiendo con más hilos?** — Porque el hardware de prueba solo tiene 4 núcleos físicos; con 8 hilos, dos comparten núcleo.
5. **¿Qué pasa si k es mayor que n?** — El código lo limita (`if (k > n) k = n`), para no crear bloques vacíos.
6. **¿Por qué se evita `sqrt` en el cálculo de distancia?** — Comparamos distancia al cuadrado contra `epsilon al cuadrado`; da exactamente la misma comparación y es más rápido (una raíz cuadrada menos por cada comparación, y se hacen muchísimas).
7. **¿Cómo generaron los puntos de entrada?** — Con `generar_puntos.py`, que reproduce exactamente el método de la libreta oficial `DBSCAN_noise.ipynb` (verificado contra su código

fuente): `sklearn.datasets.make_blobs` con 4 cúmulos gaussianos, sin agregar ruido uniforme a mano — el ruido surge naturalmente de la cola dispersa de cada cúmulo.

8. **¿Usaron IA generativa? ¿Cómo?** — Sí, para escribir el código inicial y el arnés de experimentos, bajo nuestra dirección; nosotros revisamos, corrimos y entendimos cada parte (sección de “Uso responsable y ético de IA Generativa” en el reporte).